

Placement - Verify Razorpay Buildathon

For you, I would choose **#1 AI Revenue Recovery**.

Not because it sounds coolest. Because it gives you the best combination of **AI Engineer skills + backend + agentic AI + measurable business impact + placement value**.

My ranking for you

Rank	Track	Fit for you	Why
 1	AI Revenue Recovery	9.5/10	Agentic AI + APIs + backend + business logic + measurable results
 2	AI Growth & Agentic Commerce	9/10	Excellent AI-agent exposure, but broader and potentially harder to make genuinely useful
 3	AI Finance Controller	8/10	Strong backend/data/AI engineering, but finance logic adds complexity
4	AI Risk Manager	7.5/10	ML-heavy and impressive, but evaluation/data requirements make it harder
5	Open Track	6.5/10	Maximum freedom, but also maximum opportunity to build something vague and mediocre

Why Revenue Recovery is the best choice

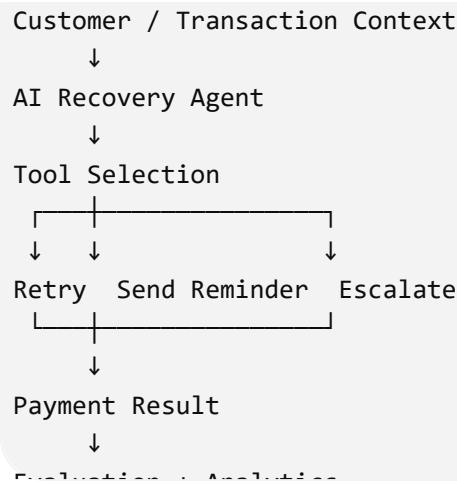
The problem naturally maps to an **AI agent**.

Imagine:

Failed payment → Agent investigates → identifies reason → decides action → executes recovery → records result

You can demonstrate an actual engineering pipeline:

```
graph TD
    A[Payment Event] --> B[Failure Classification]
    B --> C[ ]
```



That lets you showcase:

- Python
- FastAPI
- REST APIs
- SQL
- LLM integration
- Tool calling
- Agent workflows
- Razorpay APIs
- Webhooks
- Authentication
- Error handling
- Logging
- Evaluation
- Deployment
- Frontend

That's basically a miniature **AI Engineer job description disguised as a hackathon**.

Humanity does occasionally accidentally create useful things.

The biggest advantage

You can put an actual metric on the project.

For example:

"The agent recovered X% of simulated failed transactions while reducing unnecessary retries by Y%."

That's much stronger in an interview than:

"Built an AI-powered payment chatbot using Gemini."

The second sentence has been written approximately 14 billion times.

But there's one condition

Don't choose Revenue Recovery if you're going to build a simple rule-based retry system with an LLM slapped onto it.

That would be weak.

Your agent needs to **reason over transaction context and choose actions**.

For example:

```
Transaction:
₹8,500
Failure: insufficient_funds
Previous retries: 1
Customer history: 8 successful payments
Time: salary week
Preferred payment method: UPI
```

```
Agent:
→ Don't retry immediately
→ Send recovery notification
→ Offer alternate payment method
→ Retry after appropriate delay
→ Stop after defined retry limit
```

Then you evaluate whether those decisions actually improve recovery.

What about #2?

AI Growth & Agentic Commerce is your closest competitor.

I'd choose it instead if you specifically want to lean harder into **AI agents and ecommerce**.

Potential system:

```
Merchant
  ↓
Business Data
  ↓
Growth Agent
  ↓
Analyze sales/catalog/customer behavior
  ↓
Generate recommendation
```



↓
Execute action
↓
Measure revenue impact

Very good AI Engineer project.

The problem is that "**growth**" is **vague**. You need to define the business objective extremely carefully or you'll end up building an AI marketing assistant wearing a tiny agent hat.

My final recommendation

Choose:

AI Revenue Recovery

And build it as an **actual production-style AI agent**, not a college hackathon demo.

Your project should have roughly:

Frontend

→ transaction/recovery dashboard

Backend

→ FastAPI

Database

→ PostgreSQL

AI

→ LLM + structured outputs

Agent

→ LangGraph or a clean custom state-machine architecture

Payment integration

→ Razorpay APIs/webhooks

Evaluation

→ synthetic transaction dataset + recovery metrics

Deployment

→ cloud-hosted application

That project would simultaneously contribute toward your **Razorpay application, AI Engineer portfolio, backend learning, and placement preparation.**

If your goal is specifically "I want the strongest project for becoming an AI Engineer," Revenue Recovery is the one I'd pick.

This is the content they provided in the #1 track 01

AI Growth & Agentic Commerce

Grow the merchant's revenue, and make them sellable to AI buyers

Build an agent that grows revenue for a merchant on Razorpay test-mode APIs, or that makes a merchant transactable by an AI buyer end to end.

Yes. This changes my recommendation slightly, because **the actual track description is much more specific than the summary I was working from.**

You pasted **Track 01: AI Growth & Agentic Commerce**. Based on this, **I would seriously consider this as your #1 choice**, especially for your AI Engineer goal.

What Razorpay is actually asking for

The important sentence is:

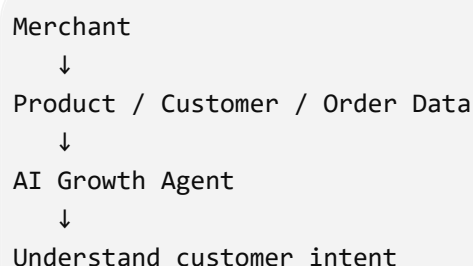
"Build an agent that grows revenue for a merchant on Razorpay test-mode APIs, or that makes a merchant transactable by an AI buyer end to end."

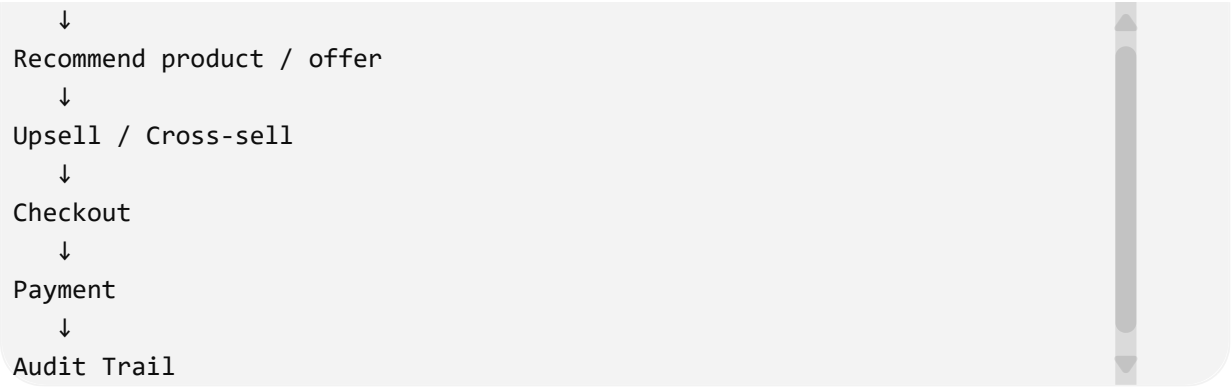
They're giving you **two possible directions**:

Option A: Revenue Growth Agent

Your agent helps a merchant make more money.

Example:





Possible implementation:

Customer:

"I need a laptop for programming under ₹70,000."

Agent:

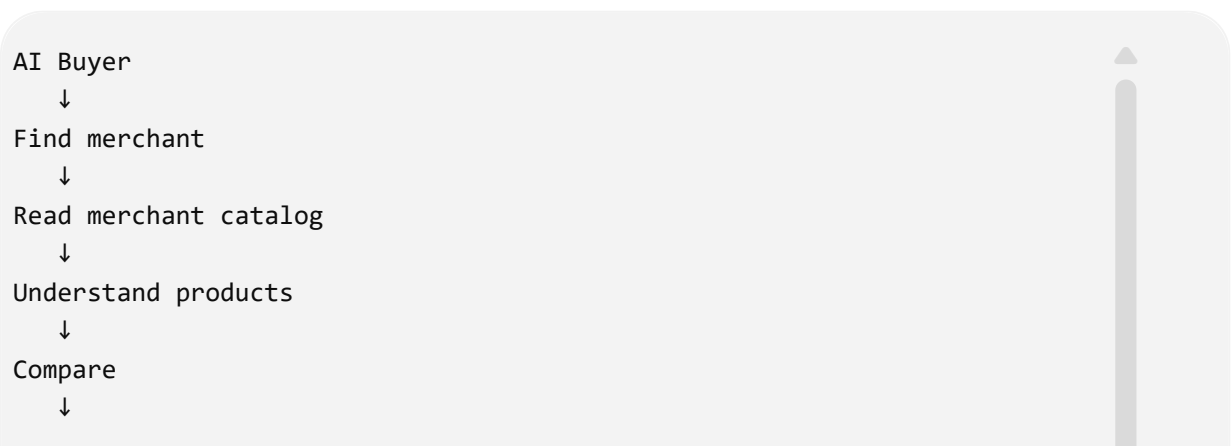
1. Understands intent
2. Searches merchant catalog
3. Finds suitable products
4. Recommends one
5. Offers relevant accessory
6. Creates checkout
7. Gets user approval
8. Initiates Razorpay test payment
9. Records everything

That's **agentic commerce**.

Option B: Make a merchant "AI-buyable"

This is actually more interesting.

Imagine an AI agent representing the **buyer**.



Select product



Create cart



Checkout



Payment



Order confirmation

The merchant's system needs to expose information in a way that another AI agent can understand and transact with.

That's much closer to the emerging **agent-to-agent commerce** problem Razorpay is talking about.

The sentence you should pay serious attention to

"Every money action explainable, bounded and gated."

This is probably one of the most important evaluation requirements.

They're **not** asking:

"Build an AI that can autonomously spend money."

That would be a spectacularly irresponsible engineering challenge.

They're asking you to build **controlled autonomy**.

For example:

AI Agent



"I want to purchase ₹4,999 headphones"



Explain:

Product: XYZ

Price: ₹4,999

Reason: matches requested features



Budget Check

$₹4,999 \leq ₹5,000$



USER APPROVAL REQUIRED



Payment



The agent should **never silently execute an unrestricted financial action.**

What "bounded and gated" means

Bounded

The agent has limits.

Maximum transaction: ₹10,000
Allowed merchants: approved merchants
Allowed payment methods: UPI/Card
Maximum retries: 2

Gated

Certain actions require explicit approval.

AI can:

- ✓ Search products
- ✓ Compare products
- ✓ Create cart
- ✓ Calculate total

AI cannot:

- X Charge money without approval
- X Exceed spending limit
- X Change payment amount

Explainable

Before a money action:

Action:
Purchase "Wireless Headphones"

Amount:
₹4,999

Reason:
Matches requested ANC + Bluetooth requirements.

Budget:
₹5,000

Approval:
Required

That's a **very strong AI engineering design.**

Why I now like Track 01 for you

You said your goal is to reach **above-intermediate AI Engineer level for placements**.

This track forces you to learn several things you actually need.

Skill	Track 01
Python	★★★★★
Backend/API	★★★★★
LLMs	★★★★★
AI Agents	★★★★★
Tool Calling	★★★★★
Database	★★★★
Authentication	★★★★
Webhooks	★★★★★
Payment systems	★★★★★
System design	★★★★★
Evaluation	★★★★★
Safety/guardrails	★★★★★

That's a **far better learning project** than another RAG chatbot.

And importantly, it gives you something interesting to explain in an interview:

"I built an AI agent capable of completing a commerce transaction through Razorpay test APIs, but designed the agent with transaction limits, explicit approval gates, explainability and an audit trail."

That is a much stronger interview conversation.

What I'd build

I wouldn't do all four example directions.

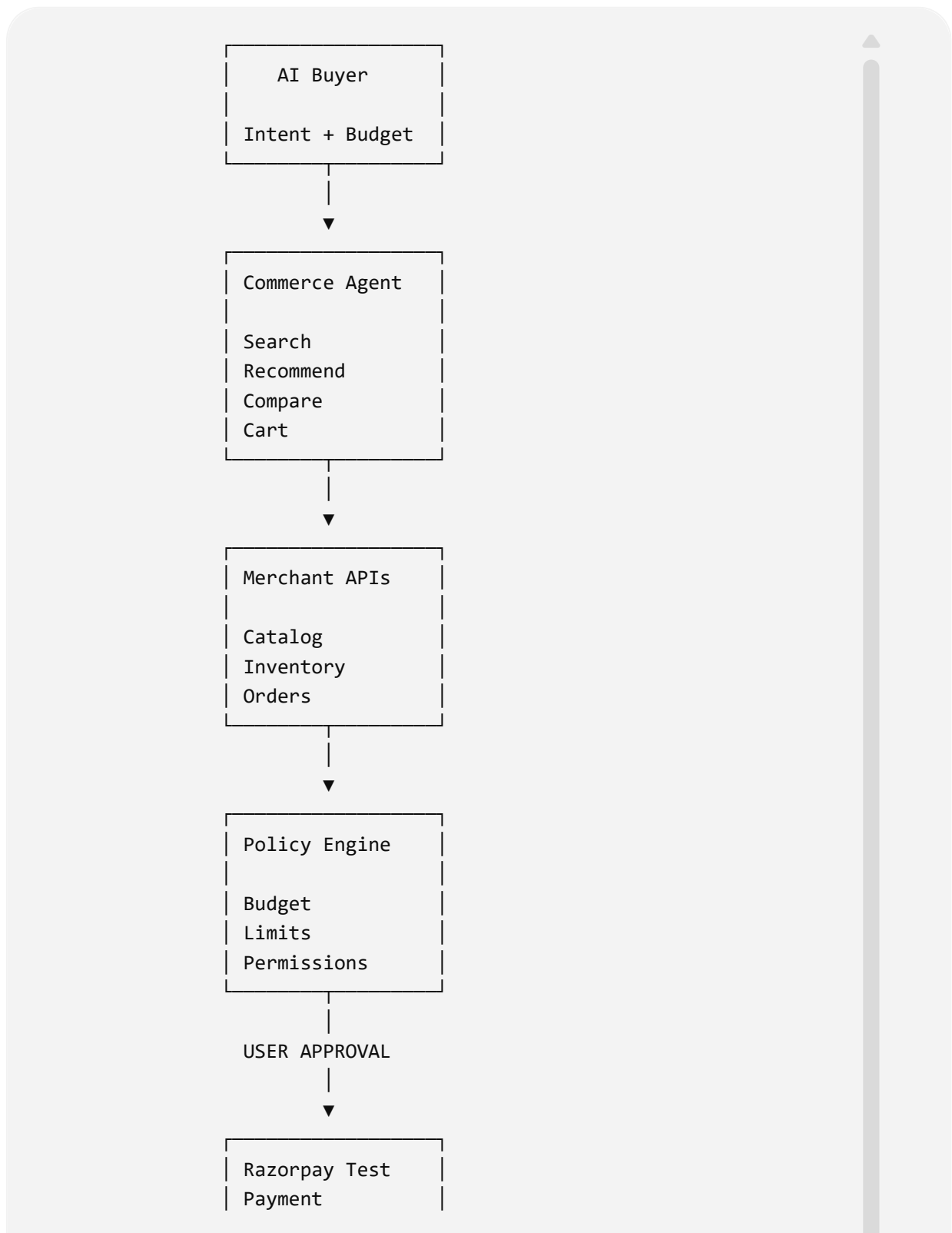
I'd pick:

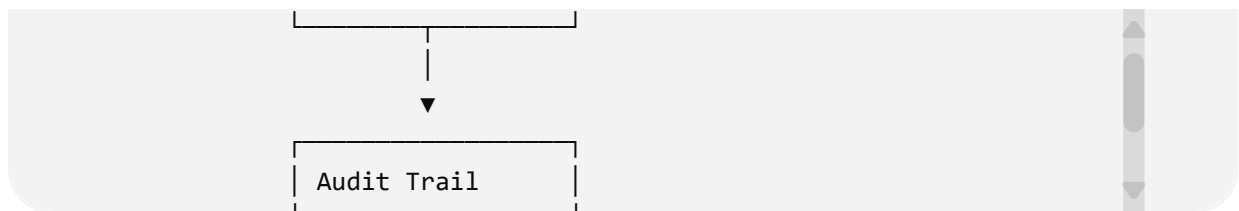
AI Buyer → Merchant → Checkout → Payment

Basically:

"AI-native merchant checkout"

Architecture:





This gives you **agent + backend + payments + security + system design + APIs** in one project.

One correction to my previous answer

I initially ranked **AI Revenue Recovery #1** based on the track summary. With the actual Track 01 requirements you've now provided, I'd change the ranking:

1. **AI Growth & Agentic Commerce** ★★★★★
2. **AI Revenue Recovery** ★★★★★
3. **AI Finance Controller** ★★★★★
4. **AI Risk Manager** ★★★★★
5. **Open Track** ★★★

For your **AI Engineer career objective**, I'd take Track 01.

The key is not to make a flashy chatbot. Build the **complete transaction loop**, and make the financial actions controlled, explainable, and auditable. That's where the project becomes genuinely impressive rather than another "LLM + Streamlit + vibes" submission.

Okay what to next

Now we stop debating tracks and **start building**. For Track 01, don't jump into coding yet. The biggest mistake would be opening VS Code and immediately installing 47 packages because apparently that's how software architecture is born.

Step 1: Lock the project direction

I recommend this specific interpretation of Track 01:

AI Buyer → Merchant → Checkout → Razorpay Payment

The core demo:

A user tells an AI buyer what they want, the agent discovers products from a merchant's catalog, recommends the best option, handles cart/checkout, asks for approval before money moves, and completes a Razorpay test-mode payment.

This directly satisfies the track:

- Agent-readable catalog ✓
 - Conversational checkout ✓
 - Upsell/cross-sell ✓
 - Merchant becomes transactable by an AI buyer ✓
 - Money action explainable ✓
 - Money action bounded ✓
 - Money action gated ✓
 - Audit trail ✓
 - Failure handling ✓
-

Step 2: Define the MVP

Don't build everything.

Your **first working version** should contain only this:

```
User
↓
AI Buyer Agent
↓
Product Search
↓
Product Recommendation
↓
Cart
↓
Order Creation
↓
Policy / Budget Check
↓
User Approval
↓
Razorpay Test Payment
↓
Success / Failure
↓
Audit Log
```

Example

User:

"I need wireless headphones under ₹5,000 with ANC."

Agent:

Searching merchant catalog...

Found 3 matching products.

Recommended:

XYZ ANC Headphones

Price: ₹4,499

ANC: Yes

Battery: 35 hours

Why:

Matches your ANC requirement and stays ₹501 under your budget.

Suggested add-on:

Carrying case - ₹399

Cart total:

₹4,898

Payment:

₹4,898

Your approval is required before payment.

User approves.

Then:

Create Razorpay order

↓

Payment

↓

Webhook

↓

Verify payment

↓

Order confirmed

↓

Audit log

That's your MVP.

Step 3: Choose the stack

For your current AI Engineer preparation, I'd use:

Backend

Python + FastAPI

AI

LLM + tool calling

You can use whichever model/API gives you reliable structured output and tool calling within the buildathon constraints.

Agent orchestration

LangGraph

But don't use LangGraph merely because it sounds AI-engineery. Use it to represent the actual state transitions.

```
graph TD;
  UNDERSTAND_INTENT --> SEARCH_PRODUCTS;
  SEARCH_PRODUCTS --> RECOMMEND;
  RECOMMEND --> BUILD_CART;
  BUILD_CART --> CHECK_POLICY;
  CHECK_POLICY --> REQUEST_APPROVAL;
  REQUEST_APPROVAL --> CREATE_PAYMENT;
  CREATE_PAYMENT --> VERIFY_PAYMENT;
  VERIFY_PAYMENT --> COMPLETE_ORDER;
```

Database

PostgreSQL

Tables roughly:

```
users
merchants
products
carts
orders
payments
agent_actions
audit_logs
```

Frontend

React

Keep it simple. You're building an engineering demo, not competing with Dribbble.

Payment

Razorpay Test Mode

This is important. Use their APIs properly rather than creating a fake payment button and calling it fintech.

Step 4: Design the agent's tools

This is where the project becomes genuinely agentic.

Your agent shouldn't directly manipulate the database however it feels like.

Give it controlled tools:

```
search_products()
get_product_details()
check_inventory()
create_cart()
add_to_cart()
calculate_total()
check_purchase_policy()
request_user_approval()
create_razorpay_order()
verify_payment()
get_order_status()
```

Then the LLM decides **which tool to call**, while your backend controls what each tool is allowed to do.

That's an important distinction:

LLM = reasoning

Backend = authority

Never give the LLM unrestricted authority over money.

Step 5: Build the safety layer

This is probably where you can differentiate your submission.

Create a Policy Engine.

Example:

```
MAX_TRANSACTION = ₹10,000  
  
MAX_DAILY_SPEND = ₹20,000  
  
PAYMENT_REQUIRES_APPROVAL = true  
  
MAX_PAYMENT_RETRIES = 2
```

Then:

```
Agent wants:  
₹14,000 purchase  
  
Policy Engine:  
❌ BLOCKED  
  
Reason:  
Transaction exceeds ₹10,000 limit.
```

The LLM cannot override this.

That's what Razorpay means by:

bounded and gated

Step 6: Build the audit trail

Every meaningful agent action should generate something like:

```
10:32:11  
USER_REQUEST  
"I need ANC headphones under ₹5,000"  
  
10:32:12  
PRODUCT_SEARCH  
Query: ANC headphones  
Budget: ₹5,000  
  
10:32:13  
RECOMMENDATION  
Product: XYZ  
Price: ₹4,499
```

```
10:32:16
CART_CREATED
Total: ₹4,898

10:32:17
POLICY_CHECK
Status: PASSED

10:32:20
PAYMENT_APPROVAL
Status: APPROVED

10:32:21
RAZORPAY_ORDER_CREATED

10:32:39
PAYMENT_VERIFIED

10:32:40
-----
```

This gives you an excellent demo screen.

Step 7: Handle one failure

The track **explicitly** asks for one failure handled gracefully.

Don't manufacture something ridiculous.

Use:

Payment failure

```
Payment attempt
  ↓
FAILED
  ↓
Agent receives failure
  ↓
Explain failure
  ↓
Suggest alternative
  ↓
User approval
  ↓
Retry / alternate payment
```

For example:

Payment failed. No money was captured. You can retry using another available

payment method.

Then log:

```
PAYMENT_FAILED
REASON: TEST_PAYMENT_FAILURE

RECOVERY_ACTION:
ALTERNATIVE_PAYMENT_METHOD

USER_APPROVAL :
REQUIRED
```

That demonstrates reliability instead of pretending production systems never break, which is a charming fantasy.

Your development order

Don't build frontend first.

Do it in this order:

Phase 1

Razorpay API understanding

↓

Phase 2

Merchant catalog API

↓

Phase 3

AI tool calling

↓

Phase 4

Agent workflow

↓

Phase 5

Policy engine

↓

Phase 6

Payment integration

↓

Phase 7

Audit trail + failure handling

↓

Phase 8

React UI

↓

Phase 9

Evaluation

↓

Phase 10

Deployment + 5-minute video

And your first task is very simple

Before writing code, we need to turn Razorpay's requirements into a **proper technical specification**.

I'll structure it into:

1. Problem statement
2. Exact MVP
3. User flow
4. System architecture
5. Agent architecture
6. Database schema
7. API endpoints
8. Agent tools
9. Safety/policy rules
10. Razorpay API integration

11. Failure scenarios
12. Evaluation metrics
13. GitHub structure
14. 5-minute demo plan

That becomes our **master blueprint**, and then we build it piece by piece instead of improvising ourselves into technical debt.